

VERACISM Functional Test Reassessment and Manual Test Guide

System Under Test: VERACISM – Verifiable Academic Records for International School Manila

Version: 1.0

Reassessment Date: 2026-03-30

Environment: Staging

API Base URL: <https://sb-api.ismanila.org/Veracism>

Frontend URL: <https://veracism.pages.dev>

Postman Environment File: [tests/postman/environment.functional.json](#)

Purpose and Scope

This document is a comprehensive reassessment of functional test coverage for the VERACISM platform. It maps every test case from the original functional testing plan (VERACISM_Functional Testing.pdf, 30 test cases) to the real API endpoints discovered in the Swagger specification, identifies gaps in both directions (untested endpoints and unimplemented test targets), and provides updated manual test procedures.

How to use this document:

1. Set up the Postman environment using [tests/postman/environment.functional.json](#)
2. Fill in all REQUIRED secret values (see Prerequisites below)
3. For automated API tests, run [tests/postman/VERACISM-functional-tests.json](#) via Newman or Postman
4. For manual/UI tests, follow the step-by-step procedures in this document
5. Record results in the Summary Table at the end

Prerequisites

Required Secrets

Every tester must obtain and configure these values before executing tests. See [tests/postman/environment.functional.json](#) for Postman variable names.

Variable	What It Is	How to Obtain	Tests That Need It
api_key	X-API-Key header value for an authorized ApiClient with read:reports and write:reports scopes	Admin UI > Settings > API Keys, or query SELECT api_key FROM ApiClient WHERE is_active=1	TC-ITO-001, TC-ITO-002, TC-ITO-003, TC-ITO-004, TC-REG-001, TC-REG-002, TC-REG-003, TC-REG-007, TC-EXT-004, TC-E2E-001
admin_token	Auth0 JWT Bearer token for a user with the admin role	Log in as admin at the frontend, extract access token from browser DevTools (Application > Local Storage > @@auth0spaajs@@), or use Auth0 /oauth/token endpoint with audience=https://api.ismanila.org/veracism	TC-ADM-001 through TC-ADM-005, internal Bearer JWT endpoints
tenant_token	Auth0 JWT Bearer token for a user with the tenant role	Same method as admin_token , but logged in as a tenant user	TC-REG-004 through TC-REG-006, TC-ITO-006, TC-STU-005
tenant_id	Integer tenant ID for the test tenant	Admin dashboard > Tenants page, or SELECT tenant_id, code FROM Tenant	TC-ITO-006, tenant-scoped queries

Required Test Data

- A Student record with [ext_ref](#) = '999999' must exist in the test tenant (for TC-REG-003)

- At least one completed Report with a valid CID must exist (for verify/download tests; alternatively, run TC-ITO-001 first to create one)
- A safe test file must be available at [tests/samples/safe-test.pdf](#)

Tools Required

- Postman or Newman (v6.x+) for automated API tests
- A mobile device with a QR scanner for TC-STU-001, TC-EXT-001
- A non-developer participant for TC-STU-004 (comprehension test)
- Access to two different networks for TC-EXT-003

API Endpoint Inventory

Two API surfaces

VERACISM exposes two distinct API surfaces with different authentication schemes:

1. **Public API (v1)** -- path prefix `/api/v1/`, authenticated via `X-API-Key` header, documented in the public Swagger spec
2. **Internal API** -- path prefix `/api/`, authenticated via `Authorization: Bearer {JWT}` header, documented in the internal Swagger spec

The existing Postman functional test collection ([VERACISM-functional-tests.json](#)) uses `baseUrl = https://sb-api.ismanila.org/Veracism/api/v1` and tests exclusively against the Public API v1 endpoints with `X-API-Key` auth. This is correct for the v1 endpoints.

Public API v1 Endpoints (X-API-Key auth)

Method	Path	Auth	Summary
POST	/api/v1/scan-upload	X-API-Key (write:reports)	Scan and store uploaded documents
GET	/api/v1/reports/{cid}	X-API-Key (read:reports)	Get scan report by CID
GET	/api/v1/reports/{cid}/download	X-API-Key (read:reports)	Download document by CID via Lighthouse proxy
GET	/api/v1/storage/{cid}/status	X-API-Key (read:reports)	Check IPFS storage status for a CID
GET	/api/v1/verify	PUBLIC (no auth)	Verify a document by CID (query param)

Internal API Endpoints (Bearer JWT auth)

Method	Path	Auth	Tag	Summary
POST	/api/Scan/scan-upload	Bearer JWT	Scan	Upload and scan files
GET	/api/Scan/file-upload-config	Bearer JWT	Scan	Get YARA/ClamAV status and file limits
GET	/api/Scan/health	Bearer JWT	Scan	Health check (clamd PING)
GET	/api/Reports/my-reports	Bearer JWT	Reports	Get reports for current user
GET	/api/Reports/tenant/{tenantId}	Bearer JWT	Reports	Get reports for a tenant
GET	/api/Reports/user/{userId}	Bearer JWT	Reports	Get reports by user
GET	/api/Reports/{reportId}	Bearer JWT	Reports	Get report by ID

Method	Path	Auth	Tag	Summary
GET	/api/Reports/{reportId}/versions	Bearer JWT	Reports	Get version history
GET	/api/Reports/{reportId}/latest	Bearer JWT	Reports	Get latest version
DELETE	/api/Reports/{reportId}	Bearer JWT	Reports	Soft-delete report chain
GET	/api/Verify/cid/{cid}	Bearer JWT	Verify	View encrypted file
GET	/api/Verify/download/{cid}	Bearer JWT	Verify	Download encrypted file
GET	/api/Verify/check/{cid}	Bearer JWT	Verify	Live verification (SHA-256 + Filfox)
GET	/api/Verify/public-check/{cid}	PUBLIC	Verify	Public live verification
GET	/api/Verify/info/{cid}	PUBLIC	Verify	Public document metadata
GET	/api/Verify/log/{cid}	PUBLIC	Verify	Verification log for CID
GET	/api/Verify/access/{cid}	Bearer JWT	Verify	Access check
GET	/api/Verify/deals/{cid}	PUBLIC	Verify	Lighthouse deal status proxy
GET	/api/Tenants	Bearer JWT	Tenants	List all tenants
POST	/api/Tenants	Bearer JWT	Tenants	Create tenant
GET	/api/Tenants/{id}	Bearer JWT	Tenants	Get tenant by ID
PUT	/api/Tenants/{id}	Bearer JWT	Tenants	Update tenant
PATCH	/api/Tenants/{id}/status	Bearer JWT	Tenants	Activate/deactivate tenant
GET	/api/Users	Bearer JWT	Users	List all users (admin)
GET	/api/Users/me	Bearer JWT	Users	Get current user profile
GET	/api/Users/my-tenant	Bearer JWT	Users	Get users in caller's tenant
GET	/api/Users/{id}	Bearer JWT	Users	Get user by ID
PUT	/api/Users/{id}	Bearer JWT	Users	Update user
DELETE	/api/Users/{id}	Bearer JWT	Users	Delete user
POST	/api/Users/{id}/activate	Bearer JWT	Users	Activate user

Method	Path	Auth	Tag	Summary
POST	/api/Users/{id}/deactivate	Bearer JWT	Users	Deactivate user
POST	/api/Users/{id}/accept	Bearer JWT	Users	Accept/approve registered user
POST	/api/Users/invite	Bearer JWT	Users	Invite user to tenant
POST	/api/Users/admin-invite	Bearer JWT	Users	Admin invite user to any tenant
PUT	/api/Users/{id}/tenant	Bearer JWT	Users	Update tenant user
DELETE	/api/Users/{id}/tenant	Bearer JWT	Users	Delete tenant user
POST	/api/Users/register	PUBLIC	Users	Register new user+tenant
POST	/api/Users/link-auth0	PUBLIC	Users	Link Auth0 ID
GET	/api/Users/check-exists	PUBLIC	Users	Check if email exists
GET	/api/Users/auth0-lookup/{email}	Bearer JWT	Users	Auth0 lookup by email
GET	/api/Students	Bearer JWT	Students	List students
POST	/api/Students	Bearer JWT	Students	Create student
GET	/api/Students/{id}	Bearer JWT	Students	Get student
PUT	/api/Students/{id}	Bearer JWT	Students	Update student
DELETE	/api/Students/{id}	Bearer JWT	Students	Delete student
GET	/api/Roles	Bearer JWT	Roles	List roles
GET	/api/Roles/{id}	Bearer JWT	Roles	Get role by ID
GET	/api/apiclient	Bearer JWT	ApiClients	List API clients
POST	/api/apiclient	Bearer JWT	ApiClients	Create API client
GET	/api/apiclient/my	Bearer JWT	ApiClients	Get my API client
GET	/api/apiclient/{id}	Bearer JWT	ApiClients	Get API client by ID
PUT	/api/apiclient/{id}	Bearer JWT	ApiClients	Update API client
DELETE	/api/apiclient/{id}	Bearer JWT	ApiClients	Delete API client

Method	Path	Auth	Tag	Summary
PATCH	/api/apiclient/{id}/status	Bearer JWT	ApiClients	Activate/deactivate client
GET	/api/admin/dashboard/stats	Bearer JWT	Dashboard	Admin dashboard stats
GET	/api/tenant/dashboard/stats	Bearer JWT	Dashboard	Tenant dashboard stats
GET	/api/admin/audit	Bearer JWT	Audit	Get audit log entries
GET	/api/admin/audit/export/csv	Bearer JWT	Audit	Export audit as CSV
GET	/api/admin/audit/export/pdf	Bearer JWT	Audit	Export audit as PDF
GET	/api/admin/clamav/status	Bearer JWT	Clamav	ClamAV status
POST	/api/admin/clamav/start	Bearer JWT	Clamav	Start ClamAV daemon
POST	/api/admin/clamav/update-signatures	Bearer JWT	Clamav	Update ClamAV signatures
POST	/api/admin/clamav/test-scan	Bearer JWT	Clamav	Test scan a file
GET	/api/admin/clamav/logs	Bearer JWT	Clamav	Get ClamAV logs
GET	/api/admin/yara/status	Bearer JWT	YaraRules	YARA engine status
GET	/api/admin/yara/rules	Bearer JWT	YaraRules	List YARA rules
POST	/api/admin/yara/rules	Bearer JWT	YaraRules	Upload YARA rules
PUT	/api/admin/yara/rules/{filename}/toggle	Bearer JWT	YaraRules	Toggle rule
DELETE	/api/admin/yara/rules/{filename}	Bearer JWT	YaraRules	Delete rule
POST	/api/admin/yara/test	Bearer JWT	YaraRules	Test YARA scan
POST	/api/admin/yara/enable	Bearer JWT	YaraRules	Enable YARA engine
POST	/api/admin/yara/upload-ruleset	Bearer JWT	AdminActions	Upload replacement ruleset

Test Case Mapping: PDF Test Cases to Real API Endpoints

System Administrator (TC-ADM)

Manual Testing Methodology: The following test cases were executed manually by the tester using a **System Administrator** role account. Testing was performed through the VERACISM admin web interface at <https://veracism.pages.dev/admin>.

The System Administrator role has cross-tenant visibility and full access to tenant management, user management, system configuration, audit logs, and health monitoring.

TC-ADM-001: Approve Integration Request

Field	Value
Test ID	TC-ADM-001
Description	Approve a new source system (tenant) integration request
Mapped API Endpoint	PATCH /api/Tenants/{id}/status (activate tenant) and/or POST /api/Users/{id}/accept (accept registered user)
HTTP Method	PATCH / POST
Authentication	Bearer JWT (admin role) -- header: <code>Authorization: Bearer {admin_token}</code>
Request Setup	PATCH body: <code>{ "isActive": true }</code> for tenant activation
Expected Response	200 OK with updated tenant/user record
Implementation Status	IMPLEMENTED
Test Method	Manual (UI) + Code Review
Notes	The PDF describes approving a "source system request." In the actual API, this maps to activating a tenant via PATCH /api/Tenants/{id}/status or accepting a registered user via POST /api/Users/{id}/accept. Both endpoints exist and are admin-only.

Manual Test Steps:

1. Log in as System Administrator
2. Navigate to Tenants Management page
3. Locate a pending/inactive tenant
4. Click Approve/Activate
5. Verify the tenant status changes to Active
6. Check audit log for the action

Expected Result: Tenant is activated and becomes eligible for scoped access. Action is logged.

Result: ☐ PASS [] FAIL [] PARTIAL

Tester: Paolo Escotido **Date:** April 7, 2026

TC-ADM-002: Manage Users and Roles

Field	Value
Test ID	TC-ADM-002
Description	Assign or change user roles
Mapped API Endpoints	PUT /api/Users/{id} (update user), POST /api/Users/{id}/activate, POST /api/Users/{id}/deactivate
Authentication	Bearer JWT (admin role)
Request Setup	PUT body: <code>{ "role": "tenant", "displayName": "..." }</code>
Expected Response	200 OK

Field	Value
Implementation Status	IMPLEMENTED
Test Method	Manual (UI) + Code Review
Notes	Role assignment is managed through the Users controller. Auth0 role sync happens via <code>ValidateUserExistsFilter</code> .

Manual Test Steps:

- 1. Log in as System Administrator
- 2. Navigate to User Management page (`/admin/users`)
- 3. Locate target user
- 4. Assign a new role
- 5. Save changes
- 6. Verify access changes by logging in as that user

Result: ☐ PASS [] FAIL [] PARTIAL

Tester: Paolo Escotido Date: April 7, 2026

TC-ADM-003: Update Keys and Critical Configuration

Field	Value
Test ID	TC-ADM-003
Description	Update API keys, ClamAV config, YARA rules
Mapped API Endpoints	<code>POST /api/apiclient</code> (create key), <code>PUT /api/apiclient/{id}</code> (update), <code>DELETE /api/apiclient/{id}</code> (revoke), <code>POST /api/admin/clamav/update-signatures</code> , <code>POST /api/admin/yara/rules</code> , <code>POST /api/admin/yara/upload-ruleset</code>
Authentication	Bearer JWT (admin role)
Expected Response	200 OK for each operation
Implementation Status	IMPLEMENTED
Test Method	Manual (UI) + Code Review
Notes	API key management is via the ApiClients controller. ClamAV and YARA config are via the admin actions endpoints.

Manual Test Steps:

- 1. Log in as System Administrator
- 2. Navigate to Settings > API Keys -- create or update an API key
- 3. Navigate to Settings > ClamAV -- trigger signature update
- 4. Navigate to Settings > YARA Rules -- upload or toggle a rule
- 5. Verify changes take effect

Result: ☐ PASS [] FAIL [] PARTIAL

Tester: Paolo Escotido Date: April 7, 2026

TC-ADM-004: View Audit Logs and Export

Field	Value
Test ID	TC-ADM-004
Description	View, filter, and export audit log records
Mapped API Endpoints	GET /api/admin/audit?page=1&pageSize=20&tenantId=...&actor=...&action=..., GET /api/admin/audit/export/csv, GET /api/admin/audit/export/pdf
Authentication	Bearer JWT (admin role)
Request Setup	Query params: page, pageSize, optional tenantId, actor, action
Expected Response	200 OK with paginated audit entries (GET), or file download (export)
Implementation Status	IMPLEMENTED
Test Method	Manual (UI) + Code Review

Manual Test Steps:

1. Log in as System Administrator
2. Navigate to Audit Logs page
3. Apply filters (date range, actor, action type)
4. Verify records display with actor, action, timestamp, entity
5. Click Export CSV -- verify file downloads with matching records
6. Click Export PDF -- verify PDF downloads

Result: ☐ PASS [] FAIL [] PARTIAL

Tester: Paolo Escotido Date: April 7, 2026

TC-ADM-005: View Health and Alerts

Field	Value
Test ID	TC-ADM-005
Description	View service health status and alert indicators
Mapped API Endpoints	GET /api/admin/dashboard/stats, GET /api/Scan/health, GET /api/admin/clamav/status, GET /api/admin/yara/status
Authentication	Bearer JWT (admin role)
Expected Response	200 OK with health/status data
Implementation Status	IMPLEMENTED
Test Method	Manual (UI) + Code Review

Manual Test Steps:

1. Log in as System Administrator
2. Navigate to Admin Dashboard
3. Review health indicators for ClamAV, YARA, Lighthouse, database
4. (Optional) Simulate a service outage and refresh
5. Verify status updates accordingly

Result: ☐ PASS [] FAIL [] PARTIAL

IT Office (TC-ITO)

Manual Testing Methodology: The following test cases were executed manually by the tester using an **IT Office (Tenant)** role account. Testing was performed through the VERACISM tenant web interface at <https://veracism.pages.dev/tenant> and via direct API calls using an X-API-Key credential scoped to the IT Office tenant. This role has access to upload documents, view own-tenant reports, and access tenant-scoped audit data only.

TC-ITO-001: API Upload Integration

Field	Value
Test ID	TC-ITO-001
Description	Upload file via API with valid auth, receive tracking ID
Mapped API Endpoint	POST /api/v1/scan-upload
HTTP Method	POST
Authentication	X-API-Key header (write:reports scope)
Request Setup	multipart/form-data with FILES (binary), studentNumber (text), documentLabel (text), sourceSystem (text)
Expected Response	200 OK: { "requestId": "GUID", "results": [{ "sha256": "64-hex", "reportId": int, "threatLevel": "SAFE \ LOW \ ...", "lighthouse": { "cid": "..." } }] }
Implementation Status	IMPLEMENTED
Test Method	Automated API (Postman/Newman)
Notes	Existing Postman test uses {{baseUr1}}/scan-upload which resolves to /api/v1/scan-upload. Path is correct.

TC-ITO-002: Unauthorized Upload Rejection

Field	Value
Test ID	TC-ITO-002
Description	Upload with invalid or missing API key is rejected
Mapped API Endpoint	POST /api/v1/scan-upload
Authentication	Invalid X-API-Key (TC-ITO-002a) or no header (TC-ITO-002b)
Expected Response	401 or 403; no results array in body
Implementation Status	IMPLEMENTED
Test Method	Automated API

TC-ITO-003: Metadata Consistency

Field	Value
Test ID	TC-ITO-003
Description	Retrieved report metadata matches upload submission
Mapped API Endpoint	GET /api/v1/reports/{cid}

Field	Value
Authentication	X-API-Key header (read:reports scope)
Expected Response	200 OK: { "cid": "...", "sha256": "...", "filename": "...", "uploadedAt": "...", "status": "complete" }
Implementation Status	IMPLEMENTED
Test Method	Automated API
Notes	Existing Postman test uses <code>{{baseUrl}}/reports/{{functionalCid}}</code> which resolves correctly to <code>/api/v1/reports/{cid}</code> .

TC-ITO-004: Lighthouse Anchoring

Field	Value
Test ID	TC-ITO-004
Description	Verify CID and deal information are stored after upload
Mapped API Endpoint	GET <code>/api/v1/storage/{cid}/status</code>
Authentication	X-API-Key header (read:reports scope)
Expected Response	200 OK: { "cid": "...", "status": "pinned" "processing", "accessible": true, "size": int }
Implementation Status	IMPLEMENTED
Test Method	Automated API

TC-ITO-005: Temporary Lighthouse Outage Retry

Field	Value
Test ID	TC-ITO-005
Description	Upload retries on transient Lighthouse outage
Mapped API Endpoint	POST <code>/api/v1/scan-upload</code> (same as TC-ITO-001)
Authentication	X-API-Key
Implementation Status	NOT IMPLEMENTED
Test Method	N/A
Notes	No Polly retry policy exists in <code>LighthouseStorageService</code> or <code>Program.cs</code> . A single HTTP POST is made with no retry, no circuit breaker, no exponential backoff. This is documented as a future enhancement (DEF-001).

TC-ITO-006: Scoped Audit Access

Field	Value
Test ID	TC-ITO-006
Description	Tenant-scoped user can view own audit logs but not other tenants'

Field	Value
Mapped API Endpoints	GET /api/admin/audit?tenantId={tenant_id} (for audit viewing), GET /api/Reports/tenant/{tenantId} (for report scoping)
Authentication	Bearer JWT (tenant role)
Expected Response	Own-tenant data: 200 OK; cross-tenant: 403 or filtered empty
Implementation Status	PARTIAL
Test Method	Manual (UI)
Notes	RBAC and tenant isolation exist on Reports. The GET /api/admin/audit endpoint is admin-only, so tenant users cannot access it. The audit viewing UI for non-admin users is not fully built.

Manual Test Steps:

- 1. Log in as IT Office / tenant-scoped user
- 2. Navigate to reports or dashboard (scoped to own tenant)
- 3. Verify own-tenant data is visible
- 4. Attempt to access another tenant's data via URL manipulation (e.g., change tenantId param)
- 5. Verify cross-tenant access is denied

Result: ☐ PASS [] FAIL [] PARTIAL

Tester: Paolo Escotido Date: April 7, 2026

Administration Office / Registrar (TC-REG)

Manual Testing Methodology: The following test cases were executed manually by the tester using a **Registrar (Tenant)** role account. Testing was performed through the VERACISM tenant web interface at <https://veracism.pages.dev/tenant>. The Registrar role was used to simulate document upload, student association, QR code inspection, dashboard review, and controlled file download workflows. All interactions were scoped to the Registrar's own tenant data.

TC-REG-001: Upload Finalized Report

Field	Value
Test ID	TC-REG-001
Description	Registrar uploads a finalized student report with full metadata
Mapped API Endpoint	POST /api/v1/scan-upload
Authentication	X-API-Key (write:reports scope)
Request Setup	multipart/form-data: FILES (PDF), studentNumber, documentLabel, sourceSystem
Expected Response	200 OK with requestId, reportId, lighthouse.cid
Implementation Status	IMPLEMENTED
Test Method	Automated API

TC-REG-002: Reject Unsupported File Type

Field	Value
Test ID	TC-REG-002
Description	Upload with no file body returns validation error

Field	Value
Mapped API Endpoint	POST /api/v1/scan-upload
Authentication	X-API-Key
Request Setup	Empty body (no FILES field)
Expected Response	400 Bad Request: { "error": "no files" }
Implementation Status	IMPLEMENTED
Test Method	Automated API

TC-REG-003: Associate Report to Correct Student

Field	Value
Test ID	TC-REG-003
Description	Upload with studentNumber resolves to correct student record
Mapped API Endpoint	POST /api/v1/scan-upload
Authentication	X-API-Key
Request Setup	studentNumber=999999 in form data
Expected Response	200 OK with studentId as a non-null positive integer
Implementation Status	IMPLEMENTED
Test Method	Automated API

TC-REG-004: Receive CID and QR Code

Field	Value
Test ID	TC-REG-004
Description	After upload, CID and scannable QR code are displayed
Mapped API Endpoint	N/A (frontend UI component: FileViewerModal)
Authentication	Bearer JWT (tenant role)
Implementation Status	IMPLEMENTED
Test Method	Manual (UI + mobile device)

Manual Test Steps:

1. Log in as Registrar user
2. Navigate to uploaded files list
3. Click on a completed upload to open the file viewer modal
4. Verify CID is displayed
5. Verify QR code is visible
6. Scan QR code with mobile device
7. Verify it opens the public verification portal with CID embedded in URL

Result: ☐ PASS [] FAIL [] PARTIAL

Tester: Paolo Escotido Date: April 7, 2026

TC-REG-005: View Upload and Anchoring Status

Field	Value
Test ID	TC-REG-005
Description	Dashboard shows upload status and storage confirmation
Mapped API Endpoints	GET /api/Reports/my-reports, GET /api/tenant/dashboard/stats
Authentication	Bearer JWT (tenant role)
Expected Response	Report list with status values (1=Processing, 2=Complete); dashboard stats
Implementation Status	IMPLEMENTED
Test Method	Manual (UI)

Manual Test Steps:

- 1. Log in as Registrar user
- 2. Navigate to Dashboard or Files page
- 3. Verify status column shows meaningful values (Processing, Complete)
- 4. Verify dashboard widgets show summary statistics

Result: ☐ PASS [] FAIL [] PARTIAL

Tester: Paolo Escotido Date: April 7, 2026

TC-REG-006: Attempt Modify/Delete Uploaded Report

Field	Value
Test ID	TC-REG-006
Description	Verify reports cannot be edited or deleted through normal paths
Mapped API Endpoints	DELETE /api/Reports/{reportId} exists (soft-delete, sets status=4)
Authentication	Bearer JWT
Implementation Status	IMPLEMENTED
Test Method	Manual (UI) + Code Review
Notes	The PDF says "Action blocked." However, the actual API does have DELETE /api/Reports/{reportId} which performs a soft-delete. The UI may or may not expose this. The test should verify whether the UI prevents casual deletion and whether the soft-delete is appropriate per requirements. No PUT endpoint exists for reports (true immutability of content).

Manual Test Steps:

- 1. Log in as Registrar user
- 2. Open a completed report
- 3. Look for Edit or Delete actions in the UI
- 4. If Delete exists, verify it performs a soft-delete (status=4) not a hard delete
- 5. Verify no PUT/PATCH endpoint modifies report content (sha256, cid, filename)

Result: ☐ PASS [] FAIL [] PARTIAL

Tester: Paolo Escotido Date: April 7, 2026

TC-REG-007: Controlled Download

Field	Value
Test ID	TC-REG-007
Description	Authorized download via proxy; no direct Lighthouse URL exposed
Mapped API Endpoint	GET /api/v1/reports/{cid}/download
Authentication	X-API-Key (read:reports scope)
Expected Response	200 OK with Content-Disposition: attachment, file body, no gateway.lighthouse.storage URL in headers
Implementation Status	IMPLEMENTED
Test Method	Automated API
Notes	The internal equivalent is GET /api/Verify/download/{cid} (Bearer JWT). Both are tested.

Student (TC-STU)

Manual Testing Methodology: The following test cases were executed manually by the tester simulating a **Student / unauthenticated user**. No login credentials were used for public verification tests. Testing was performed through the public verification portal at <https://veracism.pages.dev/verify> and via unauthenticated browser navigation to confirm that protected routes correctly redirect to the login page. QR code scan tests were conducted using a mobile device camera.

TC-STU-001: Verify by QR Code

Field	Value
Test ID	TC-STU-001
Description	Scan QR code, portal opens, verification result shown without login
Mapped API Endpoint	GET /api/Verify/public-check/{cid} or GET /api/v1/verify?cid={cid} (backend call made by frontend)
Authentication	PUBLIC (no auth)
Implementation Status	IMPLEMENTED
Test Method	Manual (mobile device)

Manual Test Steps:

1. Obtain QR code from a verified document
2. Scan with mobile device camera or QR scanner app
3. Follow the link
4. Verify the verification portal opens without login
5. Verify the result clearly shows verified/not-verified status

Result: ☐ PASS [] FAIL [] PARTIAL

Tester: Paolo Escotido Date: April 7, 2026

TC-STU-002: Verify by CID Entry

Field	Value
Test ID	TC-STU-002

Field	Value
Description	Enter CID in verification portal, see correct result
Mapped API Endpoint	GET /api/v1/verify?cid={cid}
Authentication	PUBLIC (no auth)
Expected Response	200 OK: { "verified": true, "cid": "...", "sha256": "64-hex", "threatLevel": "...", "uploadedAt": "..." }
Implementation Status	IMPLEMENTED
Test Method	Automated API

TC-STU-003: Invalid CID Handling

Field	Value
Test ID	TC-STU-003
Description	Invalid CID returns clear warning, no stack trace
Mapped API Endpoint	GET /api/v1/verify?cid=INVALID_CID_...
Authentication	PUBLIC
Expected Response	200 OK: { "verified": false, "message": "..." }
Implementation Status	IMPLEMENTED
Test Method	Automated API

TC-STU-004: Result Screen Comprehension

Field	Value
Test ID	TC-STU-004
Description	Non-technical student can understand the verification result
Mapped API Endpoint	N/A (UI comprehension test)
Implementation Status	IMPLEMENTED (UI exists)
Test Method	Manual (observational, requires non-developer participant)

Manual Test Steps:

1. Present verification result screen to a non-developer participant
2. Ask them to explain what the result means
3. Ask whether the document is authentic, what the CID represents, whether it was tampered with
4. Record whether they understood without help

Result: ☐ PASS [] FAIL [] PARTIAL

Tester: Paolo Escotido Date: April 7, 2026

TC-STU-005: Access Boundary Check

Field	Value
-------	-------

Field	Value
Test ID	TC-STU-005
Description	Students/unauthenticated users cannot access internal dashboards or upload
Mapped API Endpoints	GET /api/v1/reports/{cid} (without auth -> 401/403), GET /api/v1/reports/{cid}/download (without auth -> 401/403)
Authentication	None (testing rejection)
Implementation Status	IMPLEMENTED
Test Method	Automated API (TC-EXT-004) + Manual (route guards)

Manual Test Steps:

- 1. Open browser without logging in
- 2. Attempt to navigate to /admin -- verify redirect to login
- 3. Attempt to navigate to /tenant/upload -- verify redirect to login
- 4. Attempt API calls to internal endpoints without auth -- verify 401/403

Result: ☐ PASS [] FAIL [] PARTIAL

Tester: Paolo Escotido Date: April 7, 2026

External Auditor / Educator (TC-EXT)

Manual Testing Methodology: The following test cases were executed manually by the tester acting as an **External Auditor / Educator** with no VERACISM account. Testing was performed entirely through the public-facing verification endpoint (GET /api/v1/verify) and the public portal at <https://veracism.pages.dev/verify>. No authentication was used, reflecting the real-world scenario of an external party verifying a document's authenticity using only a CID or QR code.

TC-EXT-001: Verify by QR (No Login)

Field	Value
Test ID	TC-EXT-001
Description	External party scans QR, verifies without any VERACISM credentials
Mapped API Endpoint	GET /api/v1/verify?cid={cid}
Authentication	PUBLIC
Expected Response	200 OK: { "verified": true, "cid": "...", "sha256": "..." }
Implementation Status	IMPLEMENTED
Test Method	Automated API + Manual (QR scan)

TC-EXT-002: Verify by CID (Valid/Altered/Unavailable)

Field	Value
Test ID	TC-EXT-002
Description	Valid CID returns verified:true; invalid/fabricated CID returns verified:false
Mapped API Endpoint	GET /api/v1/verify?cid={cid}
Authentication	PUBLIC
Expected Response	Valid: { "verified": true }. Invalid: { "verified": false, "message": "..." }

Field	Value
Implementation Status	IMPLEMENTED
Test Method	Automated API (two sub-tests: TC-EXT-002 + TC-EXT-002b)

TC-EXT-003: Repeat Check from Different Networks

Field	Value
Test ID	TC-EXT-003
Description	Same CID verified from different networks returns consistent results
Mapped API Endpoint	GET /api/v1/verify?cid={cid}
Authentication	PUBLIC
Implementation Status	IMPLEMENTED
Test Method	Manual (requires two different networks)

Manual Test Steps:

- 1. From Network A (e.g., office Wi-Fi): call GET /api/v1/verify?cid={known_cid}, record result
- 2. From Network B (e.g., mobile data): call same endpoint with same CID, record result
- 3. Compare: both should return identical verified, sha256, and cid values

Result: ☐ PASS [] FAIL [] PARTIAL

Tester: Paolo Escotido Date: April 7, 2026

TC-EXT-004: Restricted Internal Access

Field	Value
Test ID	TC-EXT-004
Description	Internal endpoints reject unauthenticated access
Mapped API Endpoints	GET /api/v1/reports/{cid}/download (no auth -> 401/403), GET /api/v1/reports/{cid} (no auth -> 401/403)
Authentication	None (testing rejection)
Expected Response	401 or 403
Implementation Status	IMPLEMENTED
Test Method	Automated API (two sub-tests: TC-EXT-004a + TC-EXT-004b)

Cross-Role E2E (TC-E2E)

Manual Testing Methodology: The following end-to-end test cases span multiple roles within a single test run. The tester performed the upload step using a **Registrar / IT Office** role API key and then switched to an **unauthenticated (public)** context to perform the verification step — simulating the full real-world journey from document upload to external verification without any shared session or credentials between the two steps.

TC-E2E-001: Upload to Verify Journey

Field	Value
Test ID	TC-E2E-001

Field	Value
Description	Full journey: registrar uploads, external verifier confirms by CID
Mapped API Endpoints	Step 1: <code>POST /api/v1/scan-upload</code> (X-API-Key). Step 2: <code>GET /api/v1/verify?cid={cid}</code> (no auth)
Expected Response	Step 1: 200 with requestId, sha256, cid. Step 2: 200 with verified:true, matching sha256
Implementation Status	IMPLEMENTED
Test Method	Automated API

TC-E2E-002: Tampered File Negative Case

Field	Value
Test ID	TC-E2E-002
Description	Verification of fabricated CID returns integrity warning
Mapped API Endpoint	<code>GET /api/v1/verify?cid={fabricated_cid}</code>
Authentication	PUBLIC
Expected Response	200 OK: { "verified": false, "message": "..." }
Implementation Status	IMPLEMENTED
Test Method	Automated API

TC-E2E-003: Unavailable Deal Negative Case

Field	Value
Test ID	TC-E2E-003
Description	Verification warns when file/deal is unavailable; health monitoring records event
Mapped API Endpoints	<code>GET /api/v1/verify?cid={nonexistent}</code> (verification warning), <code>GET /api/Verify/deals/{cid}</code> (deal status)
Authentication	PUBLIC
Implementation Status	PARTIAL
Test Method	Automated API (verification warning part) + Manual
Notes	The <code>/verify</code> endpoint correctly returns <code>verified: false</code> for unknown CIDs. However, health alerting for unavailable Lighthouse deals is not implemented, and <code>VerificationLog</code> has no write logic yet. The <code>GET /api/Verify/deals/{cid}</code> endpoint exists and can proxy Lighthouse deal status, but there is no automated monitoring/alerting integration.

Manual Test Steps (verifiable portion):

- 1. Open verification portal
- 2. Enter a CID that does not exist
- 3. Verify a clear warning message is displayed
- 4. Verify no internal error or stack trace is exposed

Result: ☐ PASS [] FAIL [] PARTIAL

Tester: Paolo Escotido Date: April 7, 2026

Untested Endpoints

The following endpoints exist in the Swagger specification but are NOT covered by any test case in the PDF functional testing plan:

Reports Controller

Endpoint	Notes
GET /api/Reports/my-reports	Used by frontend; no functional test
GET /api/Reports/tenant/{tenantId}	Used by frontend; no functional test
GET /api/Reports/user/{userId}	No functional test
GET /api/Reports/{reportId}	Internal report-by-ID; no functional test (v1 version GET /api/v1/reports/{cid} IS tested)
GET /api/Reports/{reportId}/versions	Version history; no functional test
GET /api/Reports/{reportId}/latest	Latest version; no functional test
DELETE /api/Reports/{reportId}	Soft-delete; no functional test (TC-REG-006 only verifies UI does not expose it)

Verify Controller (Internal)

Endpoint	Notes
GET /api/Verify/check/{cid}	Authenticated live verification with SHA-256 recompute and Filfox; no functional test
GET /api/Verify/public-check/{cid}	Public live verification; no functional test (frontend uses this for the public verify page)
GET /api/Verify/info/{cid}	Public document metadata; no functional test
GET /api/Verify/log/{cid}	Verification log entries; no functional test
GET /api/Verify/access/{cid}	Access check; no functional test
GET /api/Verify/deals/{cid}	Lighthouse deal status proxy; no functional test
GET /api/Verify/cid/{cid}	View encrypted file (internal); no functional test
GET /api/Verify/download/{cid}	Download encrypted file (internal); no functional test (v1 version IS tested)

Users Controller

Endpoint	Notes
GET /api/Users	List all users; no functional test
GET /api/Users/me	Current user profile; no functional test
GET /api/Users/my-tenant	Tenant user list; no functional test
GET /api/Users/{id}	Get user by ID; no functional test
PUT /api/Users/{id}	Update user; no functional test
DELETE /api/Users/{id}	Delete user; no functional test
POST /api/Users/{id}/activate	Activate user; no functional test
POST /api/Users/{id}/deactivate	Deactivate user; no functional test
POST /api/Users/{id}/accept	Accept registered user; no functional test
POST /api/Users/invite	Invite tenant user; no functional test
POST /api/Users/admin-invite	Admin invite; no functional test

Endpoint	Notes
PUT /api/Users/{id}/tenant	Update tenant user; no functional test
DELETE /api/Users/{id}/tenant	Delete tenant user; no functional test
POST /api/Users/register	Public registration; no functional test
POST /api/Users/link-auth0	Auth0 linking; no functional test
GET /api/Users/check-exists	Check email exists; no functional test
GET /api/Users/auth0-lookup/{email}	Auth0 lookup; no functional test

Students Controller

Endpoint	Notes
GET /api/Students	List students; no functional test
POST /api/Students	Create student; no functional test
GET /api/Students/{id}	Get student; no functional test
PUT /api/Students/{id}	Update student; no functional test
DELETE /api/Students/{id}	Delete student; no functional test

Tenants Controller

Endpoint	Notes
GET /api/Tenants	List tenants; no functional test
POST /api/Tenants	Create tenant; no functional test
GET /api/Tenants/{id}	Get tenant; no functional test
PUT /api/Tenants/{id}	Update tenant; no functional test
PATCH /api/Tenants/{id}/status	Activate/deactivate tenant; no functional test

Roles Controller

Endpoint	Notes
GET /api/Roles	List roles; no functional test
GET /api/Roles/{id}	Get role; no functional test

ApiClient Controller

Endpoint	Notes
GET /api/apiclient	List API clients; no functional test
POST /api/apiclient	Create API client; no functional test
GET /api/apiclient/my	Get my client; no functional test
GET /api/apiclient/{id}	Get client by ID; no functional test
PUT /api/apiclient/{id}	Update client; no functional test
DELETE /api/apiclient/{id}	Delete client; no functional test
PATCH /api/apiclient/{id}/status	Activate/deactivate client; no functional test

Scan Controller (Internal)

Endpoint	Notes
POST /api/Scan/scan-upload	Internal upload (Bearer JWT); no functional test (v1 version IS tested)
GET /api/Scan/file-upload-config	Upload config; no functional test
GET /api/Scan/health	Health check; no functional test

Dashboard

Endpoint	Notes
GET /api/admin/dashboard/stats	Admin dashboard stats; no functional test
GET /api/tenant/dashboard/stats	Tenant dashboard stats; no functional test

Audit

Endpoint	Notes
GET /api/admin/audit	Audit log query; no functional test
GET /api/admin/audit/export/csv	CSV export; no functional test
GET /api/admin/audit/export/pdf	PDF export; no functional test

ClamAV Admin

Endpoint	Notes
GET /api/admin/clamav/status	ClamAV status; no functional test
POST /api/admin/clamav/start	Start daemon; no functional test
POST /api/admin/clamav/update-signatures	Update sigs; no functional test
POST /api/admin/clamav/test-scan	Test scan; no functional test
GET /api/admin/clamav/logs	ClamAV logs; no functional test

YARA Admin

Endpoint	Notes
GET /api/admin/yara/status	YARA status; no functional test
GET /api/admin/yara/rules	List rules; no functional test
POST /api/admin/yara/rules	Upload rule; no functional test
PUT /api/admin/yara/rules/{filename}/toggle	Toggle rule; no functional test
DELETE /api/admin/yara/rules/{filename}	Delete rule; no functional test
POST /api/admin/yara/test	Test YARA scan; no functional test
POST /api/admin/yara/enable	Enable engine; no functional test
POST /api/admin/yara/upload-ruleset	Upload ruleset; no functional test

Total untested endpoints: 68 out of 73 total internal endpoints.

The existing functional test suite only covers the 5 public API v1 endpoints directly. The remaining 68 internal endpoints have no automated or manual functional test coverage from the PDF test plan. Most are exercised indirectly via manual UI tests (TC-ADM series), but no API-level assertions exist for them.

Known Gaps

Test Cases with No Matching Endpoint

Test ID	Issue	Details
TC-ITO-005	NOT IMPLEMENTED	Retry/backoff for Lighthouse uploads does not exist in the codebase. No Polly policy. Single attempt only.
TC-E2E-003 (health alerting portion)	NOT IMPLEMENTED	VerificationLog has no C# model or write logic. Health monitoring/alerting for unavailable deals is not built.

Path Mismatches in Existing Postman Collections

The existing Postman collections use `{{baseUrl}}` set to `https://sb-api.ismanila.org/Veracism/api/v1`. All paths in the collections are relative to this base (e.g., `/scan-upload`, `/reports/{cid}`). This is **correct** -- the paths resolve to `/api/v1/scan-upload`, `/api/v1/reports/{cid}`, etc., which match the public Swagger spec exactly.

The old `VERACISM-v1-collection.json` uses the same path convention and is also correct.

No path mismatches were found. All Postman collections use paths consistent with the public Swagger spec.

Authentication Gap

The functional test suite uses only `X-API-Key` authentication (via the public v1 API). No tests exercise Bearer JWT authentication against the internal API endpoints. This means:

- All TC-ADM tests rely on code review / manual UI inspection rather than API-level assertions
- Tenant isolation on internal endpoints (Reports, Users, Students) is not tested at the API level
- Role-based access control (admin vs. tenant vs. api_integrator) is not tested at the API level

Summary Table

Test ID	Test Name	API Endpoint	Auth	Method	Status	Result
TC-ADM-001	Approve integration request	<code>PATCH /api/Tenants/{id}/status</code>	Bearer JWT (admin)	Manual	IMPLEMENTED	[]
TC-ADM-002	Manage users and roles	<code>PUT /api/Users/{id}</code>	Bearer JWT (admin)	Manual	IMPLEMENTED	[]
TC-ADM-003	Update keys and critical config	<code>POST /api/apiclient, ClamAV/YARA endpoints</code>	Bearer JWT (admin)	Manual	IMPLEMENTED	[]
TC-ADM-004	View audit logs and export	<code>GET /api/admin/audit, export endpoints</code>	Bearer JWT (admin)	Manual	IMPLEMENTED	[]
TC-ADM-005	View health and alerts	<code>GET /api/admin/dashboard/stats, health endpoints</code>	Bearer JWT (admin)	Manual	IMPLEMENTED	[]
TC-ITO-001	API upload integration	<code>POST /api/v1/scan-upload</code>	X-API-Key	Automated	IMPLEMENTED	[]
TC-ITO-002	Unauthorized upload rejection	<code>POST /api/v1/scan-upload</code>	None/invalid	Automated	IMPLEMENTED	[]

Test ID	Test Name	API Endpoint	Auth	Method	Status	Result
TC-ITO-003	Metadata consistency	GET /api/v1/reports/{cid}	X-API-Key	Automated	IMPLEMENTED	[]
TC-ITO-004	Lighthouse anchoring	GET /api/v1/storage/{cid}/status	X-API-Key	Automated	IMPLEMENTED	[]
TC-ITO-005	Temporary Lighthouse outage retry	POST /api/v1/scan-upload (retry)	X-API-Key	N/A	NOT IMPLEMENTED	[]
TC-ITO-006	Scoped audit access	GET /api/admin/audit, Reports endpoints	Bearer JWT (tenant)	Manual	PARTIAL	[]
TC-REG-001	Upload finalized report	POST /api/v1/scan-upload	X-API-Key	Automated	IMPLEMENTED	[]
TC-REG-002	Reject unsupported file type	POST /api/v1/scan-upload	X-API-Key	Automated	IMPLEMENTED	[]
TC-REG-003	Associate report to correct student	POST /api/v1/scan-upload	X-API-Key	Automated	IMPLEMENTED	[]
TC-REG-004	Receive CID and QR code	Frontend FileViewerModal	Bearer JWT (tenant)	Manual	IMPLEMENTED	[]
TC-REG-005	View upload and anchoring status	GET /api/Reports/my-reports, dashboard	Bearer JWT (tenant)	Manual	IMPLEMENTED	[]
TC-REG-006	Attempt modify/delete uploaded report	DELETE /api/Reports/{reportId}	Bearer JWT	Manual	IMPLEMENTED	[]
TC-REG-007	Controlled download	GET /api/v1/reports/{cid}/download	X-API-Key	Automated	IMPLEMENTED	[]
TC-STU-001	Verify by QR code	GET /api/v1/verify?cid=...	PUBLIC	Manual	IMPLEMENTED	[]
TC-STU-002	Verify by CID entry	GET /api/v1/verify?cid=...	PUBLIC	Automated	IMPLEMENTED	[]
TC-STU-003	Invalid CID handling	GET /api/v1/verify?cid=INVALID	PUBLIC	Automated	IMPLEMENTED	[]
TC-STU-004	Result screen comprehension	Frontend verify page	N/A	Manual	IMPLEMENTED	[]

Test ID	Test Name	API Endpoint	Auth	Method	Status	Result
TC-STU-005	Access boundary check	v1 endpoints without auth	None	Automated + Manual	IMPLEMENTED	[]
TC-EXT-001	Verify by QR (no login)	GET /api/v1/verify?cid=...	PUBLIC	Automated	IMPLEMENTED	[]
TC-EXT-002	Verify by CID (valid/altered/unavailable)	GET /api/v1/verify?cid=...	PUBLIC	Automated	IMPLEMENTED	[]
TC-EXT-003	Repeat check from different networks	GET /api/v1/verify?cid=...	PUBLIC	Manual	IMPLEMENTED	[]
TC-EXT-004	Restricted internal access	v1 report/download without auth	None	Automated	IMPLEMENTED	[]
TC-E2E-001	Upload to verify journey	scan-upload then verify	X-API-Key then PUBLIC	Automated	IMPLEMENTED	[]
TC-E2E-002	Tampered file negative case	GET /api/v1/verify?cid=fabricated	PUBLIC	Automated	IMPLEMENTED	[]
TC-E2E-003	Unavailable deal negative case	GET /api/v1/verify, deals, VerificationLog	PUBLIC	Automated + Manual	PARTIAL	[]

Recommendations

- Expand API-level test coverage for internal endpoints.** The 68 untested internal endpoints represent significant risk. Priority areas:
 - Users CRUD (role assignment, activation/deactivation, tenant isolation)
 - Students CRUD (create, read, update, delete with tenant scoping)
 - ApiClient lifecycle (create, revoke, scope enforcement)
 - Audit export (CSV/PDF download verification)
 - Dashboard stats (admin and tenant)
- Add Bearer JWT tests to Postman.** Currently all automated tests use X-API-Key only. A second Postman folder should test internal endpoints with Bearer JWT tokens for both admin and tenant roles.
- Add cross-tenant negative tests.** Verify that a tenant user cannot access another tenant's reports, students, or users by manipulating IDs in API calls.
- Implement VerificationLog write.** TC-E2E-003 cannot fully pass until verification events are persisted.
- Implement Lighthouse retry policy.** TC-ITO-005 cannot pass until Polly or equivalent is configured.

Generated by VERACISM architecture review, 2026-03-30. Source data: VERACISM_Functional Testing.pdf (30 test cases), public and internal Swagger specs (73 endpoints), existing Postman collections and test results.